

- создать программу игрока, обеспечивающего передачу паса;
- создать программу игрока, обеспечивающего забивание гола, после переданного паса, с учетом возможных погрешностей в передаче паса;
- обеспечить многократное повторение голевого паса в течение игры.

4.2. Основные теоретические сведения

Аудиосообщения. Информация об изменениях в ходе игры предоставляется рефери в виде аудиосообщений `hear`. Формат сообщения: (`hear` Время Источник Текст). Здесь Время – такт, на котором передано сообщение, Источник – отправитель сообщения (если игрок сам отправил данное сообщение, то Источник будет `self`), Текст – текст сообщения. Важно, что текст сообщения, переданный игроком `say`, будет заключен в кавычки:

```
(say go)
(hear 100 self "go")
(hear 110 referee play_on)
```

Аудиосообщения могут использоваться для координации действий игроков, если они находятся не слишком далеко друг от друга. В частности, аудиосообщение рефери `play_on` гласит, что игра начата (в самом начале игры и после забитого мяча), а сообщение `goal_1_2` гласит, что команда слева забила второй гол.

Соответственно, агент может в дереве решений учитывать полученные аудиосообщения. Например, при получении сообщения `play_on` можно начинать движение (`play_on` в течение игры может быть отправлен несколько раз), а после получения сообщения, начинающегося на «`goal_`», может с использованием команды (`move x y`) переместиться в начальное положение (`x, y`) на поле.

В начале игры (перед получением сообщения `play_on`) и после каждого забитого гола (после «`goal_...`») мяч оказывается в центре поля, что позволяет игрокам начать целевую последовательность действий для передачи голевого паса и забивания гола с начала.

При передаче голевого паса может сложиться ситуация, когда игрок окажется близко к воротам противника в отсутствие игроков противника, что может привести к состоянию вне игры (`offside`). В связи с этим целесообразно «установить» двух противников у ворот, например около каждой штанги. Эти два игрока противника должны стоять на месте независимо от сообщений сервера.

Отличия игрока своей команды. При передаче паса необходимо проверить команду, к которой относится игрок, принимающий пас. В сообщениях от сервера `see` игрок команды `teamA` в зависимости от расстояния до него может обозначаться как `p_"teamA"_N`, где `N` – номер игрока, `p_"teamA"` или даже просто `p` (примеры приведены по возрастанию дистанции до видимого игрока).

Игрок, передающий пас. Для игрока, передающего голевой пас, возможны следующие ситуации после приближения к мячу:

1. Игрок, забивающий гол, виден, тогда ему передается пас.
2. Игрок, забивающий гол, не виден, тогда в течение `N` тактов игры выполняется ожидание выполнения условий п. 1, после `N` тактов выполняется поворот с мячом (серия слабых ударов под углом) до тех пор, пока не будут выполнены условия п. 1.

Игрок, забивающий гол. Для игрока, забивающего гол, возможны следующие ситуации после передачи паса:

1. Игрок увидел мяч, приблизился к нему на расстояние удара и видит центр ворот `gr` (правые ворота), тогда выполняет удар по мячу в сторону `gr`.
2. Игрок увидел мяч, приблизился к нему на расстояние удара и не видит центр ворот `gr`, в таком случае выполняется поворот с мячом (серия слабых ударов под углом 45°) до тех пор, пока в поле зрения не появится `gr`, затем выполняется удар по мячу в сторону `gr`.
3. Игрок не увидел мяч впереди себя (мяч передан на недостаточное расстояние), в таком случае игрок продолжает движение до целевого флага (или до заданного расстояния до него), затем разворачивается и начинает искать переданный мяч.

4.3. Вопросы для самопроверки

1. В чем отличия аудиосообщений от игрока и от рефери?
2. Как отличить игрока своей команды от игрока команды противника по данным в сообщении `see`?
3. В какие периоды сервер виртуального футбола позволяет использовать команду `move`?

4.4. Задание

Необходимо разработать с использованием деревьев решений две программы (два дерева), имитирующие игрока, дающего голевой пас, и игрока, забивающего гол.

Программа игрока, дающего голевой пас, должна решать следующие задачи:

1. После сообщения рефери `play_on` начать движение в сторону флага `fplc`, а затем к мячу.
2. После приближения к мячу на расстояние удара дать пас игроку, забивающему гол, и отправить аудиосообщение "go".
3. После паса ожидание сообщения «goal...» или движение к флагу `fplc`.
4. После сообщения «goal...» возврат в исходную позицию и ожидание сообщения `play_on`.

Программа игрока, забивающего гол, должна решать следующие задачи:

1. После сообщения рефери `play_on` начать движение в сторону флага `fplb` (или `fplt`), а затем к `fgrb` (или `fgrt`).
2. После аудиосообщения "go" продолжить движение прямо, но ожидать получения паса (появления мяча в зоне видимости).
3. При появлении мяча в зоне видимости или после достижения соответствующего флага и возврата к мячу выполнить удар по воротам `gr`.
4. После сообщения «goal...» возврат в исходную позицию и ожидание сообщения `play_on`.

В результате в течение игры должно быть забито не менее 10 голов.

Практическая работа 5. РЕШЕНИЕ ЦЕЛЕВОЙ ЗАДАЧИ В УСЛОВИЯХ ПРОТИВОДЕЙСТВИЯ

5.1. Цель и задачи

Целью работы является решение задачи создания универсального механизма управления игроком в мультиагентной среде.

Для достижения поставленной цели необходимо решить следующие задачи:

- создать программу, обеспечивающую интерпретацию временного автомата;
- создать временные автоматы для игроков и вратаря;
- проверить забивание гола и защиту ворот от забивания гола (должны быть и успешные перехваты, и успешные голы).

5.2. Основные теоретические сведения

Временной автомат — это конечный автомат, расширенный набором вещественных таймеров (часов). При выполнении временного автомата значения всех таймеров увеличиваются с одинаковой скоростью. При выполнении переходов значения таймеров могут сравниваться с целочисленными константами, используемыми в качестве инвариантов (условий перехода), и, соответственно, разрешать или запрещать переходы. Значения таймеров могут сбрасываться.

С математической точки зрения временной автомат — это система (L, l_0, A, C, P, E) , в которой L — конечное множество состояний, l_0 — начальное состояние ($l_0 \in L$), A — конечное множество действий (алфавит автомата), C — конечное множество таймеров, $P: L \rightarrow TC$ — множество допустимых состояний с учетом значения всех таймеров и соответствующих инвариантов, $E: L \times TC \times A \times R(C) \times L$ — множество переходов. Каждый переход представляет собой кортеж $\langle l_1, g, a, r, l_2 \rangle$ — переход из состояния l_1 в состояние l_2 с выполнением действия a , предусловием g и множеством сбрасываемых часов r .

Классический конечный автомат начинает работу в начальном состоянии, последовательно считывает по одному символу входного слова (цепочки входных символов), при этом считанный символ переводит автомат в новое состояние в соответствии с функцией переходов. Для формулирования состояния временного автомата и возможных переходов необходимо не только знать текущее состояние, но и значения таймеров, что описывается кортежем $\langle l, v \rangle$, где l — состояние, $v: C \rightarrow R^+$ — вещественные значения таймеров. В начальном состоянии $\langle l_0, v_0 \rangle: v_0(c) = 0$ для всех таймеров c . Шаг вычисления временного автомата состоит в продвижении времени ($\langle l, v \rangle \rightarrow \langle l, v+d \rangle$) и выполнении перехода ($\langle l, v \rangle \rightarrow \langle l', v' \rangle$). При продвижении времени не выполняется никакое действие, а значение всех таймеров увеличивается на заданную константу. Для выполнения перехода проверяются инварианты, выбирается один из возможных переходов, выполняется переход в новое состояние, при необходимости сбрасываются указанные таймеры.

Кортеж $\sigma = \langle l, v \rangle$ называют конфигурацией временного автомата. Трассой автомата, исходящей из конфигурации σ , называется последовательность

конфигураций вида $\sigma \rightarrow \sigma_1 \rightarrow \sigma_2 \rightarrow \dots$. Конфигурация называется тупиковой, если не существует конфигурации σ' , такой что $\sigma \rightarrow \sigma'$. Трасса называется полной, если она бесконечна или оканчивается тупиковой конфигурацией. Вычислением автомата называется формирование полной трассы, исходящей из начальной конфигурации. По аналогии с конечными автоматами временные автоматы могут быть детерминированными и недетерминированными.

В теории временных автоматов предусмотрено одновременное выполнение нескольких автоматов, при этом действия синхронизации делятся на два типа: активные (при обозначении добавляется восклицательный знак $a!$) и пассивные (при обозначении добавляется знак вопроса $a?$). Они соответствуют отправке и приему сигнала a . При выполнении перехода с синхронизацией соответствующие действия в обоих автоматах выполняются одновременно. Таким образом может быть сформирована сеть автоматов, взаимодействующих друг с другом.

Uppaal. Временные автоматы исследуются ведущими университетами мира и существуют бесплатные реализации для некоммерческого использования, например Uppaal (www.uppaal.org), разработанная совместно Uppsala Universitet (www.uu.se/en/) и Aalborg University (www.en.aau.dk).

Страница для скачивания:

<http://www.it.uu.se/research/group/darts/uppaal/download.shtml>.

Руководство пользователя:

<http://www.it.uu.se/research/group/darts/papers/texts/new-tutorial.pdf>.

Краткое руководство:

http://www.it.uu.se/research/group/darts/uppaal/small_tutorial.pdf.

Uppaal предусматривает реализацию сети автоматов. Каждый проект включает в себя объявления переменных, набор шаблонов временных автоматов (не менее одного) и системные объявления, в которых регистрируются используемые конечные автоматы.

Рассмотрим базовые возможности Uppaal (более подробно – в документации).

Вкладка «Редактор». Объявление переменных. Каждое объявление заканчивается точкой с запятой.

Для объявления константы используется ключевое слово `const`, тип константы (`int`, `bool`), имя и значение. Пример: `const int N = 50;`

Для объявления переменной из заданного интервала указывается тип переменной, в квадратных скобках через запятую интервал значений, затем имя переменной и начальное значение. Пример: `int[0,N] dist = 25;`

Для объявления массива указывается тип переменных в массиве, имя массива и затем в квадратных скобках размер массива. Пример: `int a[5];`

Для указания действий в автомате используется ключевое слово `chan`, за которым через запятую перечисляются все действия, обрабатываемые автоматом (как активные, так и пассивные). Пример: `chan back, kick, run;`

Добавление и удаление шаблонов осуществляется с использованием меню «Правка»: «Правка → Вставить шаблон», «Правка → Удалить шаблон». Для переименования шаблон достаточно выделить и нажать клавишу «F2» или изменить в поле «Имя:». В поле «Параметры:» можно указать параметры, передаваемые в автомат.

У каждого шаблона могут быть свои определения (раскрыть узел с именем шаблона автомата и выбрать дочерний узел «Определения»). Обычно в них задают таймеры, используемые в шаблоне автомата. Пример: `clock t;`

В системных объявлениях задают перечень шаблонов, используемых для симуляции. После ключевого слова `system` через запятую перечисляют имена шаблонов. Пример: `system Goalie, Engine;`

Редактирование шаблона временного автомата осуществляется с использованием графического интерфейса. Кружками обозначены узлы. Двойной круг – начальный узел (начальное состояние), круг с буквой «С» – непрерывный узел. Состояния могут иметь имена, которые отображаются рядом с узлами коричневым цветом.

Переходы между состояниями отображаются с использованием стрелок (прямых или изогнутых). При двойном щелчке мышью на стрелках открывается меню редактирования свойств перехода. Можно указать предусловие, например `t <= 10`, выполнить действие синхронизации, например «kick!» или «kick?» (далее будут приводиться без кавычек), выполнить обновление значений переменных и таймеров, например `t = 0`. Предусловия, действия и правила обновления значений отображаются рядом со стрелками.

Все надписи и узлы могут перемещаться по графу. Изгиб стрелок может быть изменен после их добавления.

Вкладки «Симулятор» и «Конкретный симулятор». Если при переходе на вкладки «Симулятор» и «Конкретный симулятор» выдаются ошибки, то их необходимо устранить во вкладке «Редактор».

В списке «Доступные переходы» вкладки «Симулятор» приведены доступные переходы. Для симуляции необходимо выбрать один из доступных переходов и нажать кнопку «Вперед». Сброс симуляции осуществляется по кнопке «Сброс», в списке «Симуляция Трассы» приводятся все состояния автомата в процессе симуляции. На вкладке в уменьшенном виде приводятся все автоматы, красным цветом подсвечены текущие состояния, в списке «Global Variables» приведены глобальные переменные, в списке «Constraints» приведены инварианты. При нажатии кнопки «Случайно» будет выполняться симуляция со случайным выбором действий.

На вкладке «Конкретный симулятор» у пользователя есть возможность не только выбрать действие, но и задержку во времени до ее выполнения.

Вратарь. Рассмотрим пример применения временных автоматов к управлению вратарем в игре «виртуальный футбол». Для простейшего автомата потребуются следующие объявления переменных:

```
const int N = 70;  
int[0,N] dist = 25;  
chan goBack, kick, runToBall, lookAround, canIntercept, catch,  
ok;
```

Здесь константа N указывает возможное расстояние до мяча, переменная $dist$ – текущее расстояние до мяча, приведен перечень действий синхронизации.

Для реализации автомата вратаря потребуются два шаблона: шаблон действий вратаря *Goalie* и шаблон, имитирующий поведение окружающей среды *Engine*, поэтому системные объявления будут выглядеть следующим образом:

```
system Goalie, Engine;
```

Простейший автомат для вратаря представлен на рис. 5.1. В данном конечном автомате вратарь оценивает расстояние до мяча (*Close*, *Near*, *Far*) с использованием значения переменной $dist$. Переход в состояние *Close* возможен только при условии $dist < 2$, в состояние *Far* – $dist \geq 10$, *Near* – остальные варианты.

Из состояния *Close* осуществляются последовательные переходы в *Catch* (с выполнением синхронизирующего действия *catch!*), в *Kick* (с выполнением синхронизирующего действия *kick!*) и обратно в *Start* (с выполнением синхронизирующего действия *goBack!* и сбрасыванием таймера $t=0$).

Из состояния *Far* есть два перехода в состояние *Start*: при условии $t \leq 10$ (с выполнением синхронизирующего действия *ok!*) и при условии

$t > 10$ (с выполнением синхронизирующего действия `lookAround!` и сбрасыванием таймера $t=0$).

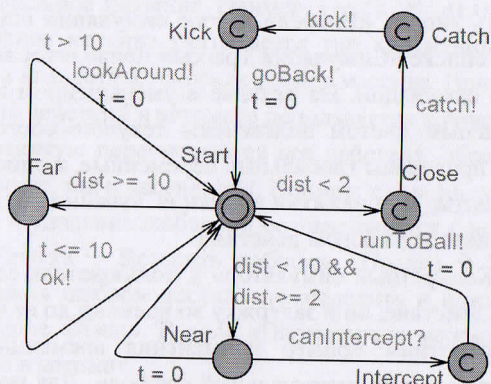


Рис. 5.1. Временной автомат вратаря

Из состояния `Near` есть два перехода: один в состояние `Start` (со сбрасыванием таймера $t=0$) и в состояние `Intercept` (при получении синхронизирующего сообщения `canIntercept?`), после которого выполняется переход в состояние `Start` (с выполнением синхронизирующего действия `runToBall!` и сбрасыванием таймера $t=0$).

Для корректной работы временному автомату нужен второй автомат (рис. 5.2), который будет играть роль окружающей среды, будет обрабатывать сообщения от вратаря и генерировать реакции от окружающей среды (в частности, изменять значение переменной `dist`).

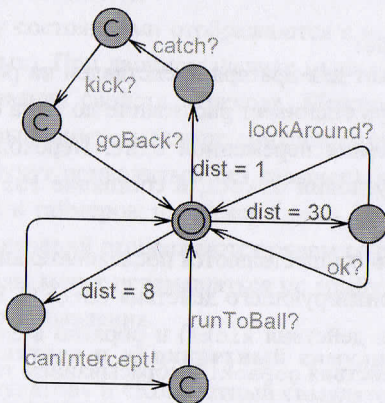


Рис. 5.2. Временной автомат окружающей среды

Обратите внимание, что у каждого синхронизирующего действия есть пара ($a!$ – $a?$).

Реализация временного автомата. Временной автомат должен оперировать не сообщениями о том, что видит и слышит игрок, а данными, подготовленными для обработки, такими как:

- текущее и предыдущее положения мяча с указанием координат, угла, расстояния до мяча;
- игроки своей команды и команды противника с указанием координат, угла и расстояния;
- ворота свои и чужие с указанием угла и расстояния до них;
- свои координаты;
- текущее время;
- история аудиосообщений с указанием времени, текста и отправителя;
- значения таймеров и переменных.

Перечисленные значения получаются с использованием готовых стандартных функций, обеспечивающих вычисление расстояния между объектами (игроками, мячом), определение объектов, ближайших к заданному (например, к мячу), возможность сбрасывать значения таймеров.

Пример данных, воспринимаемых игроком, в формате JSON (JavaScript Object Notation):

```
{
  "time":135,
  "pos":{"x":-44.77, "y":-0.48},
  "hear":[{"time":100, "who":"referee", "msg":"play_on"},
    {"time":100, "who":"referee", "msg":"drop_ball"},
    {"time":0, "who":"referee", "msg":"kick_off_1"}],
  "ballPrev":{"x":-44, "y":-0.7, "f":"b", "dist":0.7, "angle":6},
  "ball":{"x":-44.3, "y":-0.6, "f":"b", "dist":0.5, "angle":8},
  "teamOwn":[],
  "team":[{"x":-27.2, "y":-5.5, "f":"p\\teamA\\1", "dist":18.2,
    "angle":17}],
  "goal":{"x":0, "y":0.72, "f":"gr", "dist":97.5, "angle":2}
}
```

В приведенном примере объект имеет следующие атрибуты: `time` – текущее время; `pos` – вычисленная позиция игрока; `hear` – упорядоченный список всех сообщений, услышанных игроком; `ball` – положение мяча; `ballPrev` – предыдущее положение мяча; `teamOwn` – список видимых игроков своей команды; `team` – список видимых игроков команды противника; `goal` – ворота противника; `goalOwn` – свои ворота.

Представленный временной автомат с учетом предложенного формата хранения данных может быть описан следующим образом:

```
const TA = {
  current: "start", // Текущее состояние автомата
  state: { // Описание состояния
    variables: { dist: null }, // Переменные
    timers: { t: 0 }, // Таймеры
    next: true, // Нужен переход на следующее состояние
    synch: undefined, // Текущее действие
    local: {}, // Внутренние переменные для методов
  },
  nodes: { /* Узлы автомата, в каждом узле: имя и узлы, на кото-
    рые есть переходы */
    start: { n: "start", e: ["close", "near", "far"] },
    close: { n: "close", e: ["catch"] },
    catch: { n: "catch", e: ["kick"] },
    kick: { n: "kick", e: ["start"] },
    far: { n: "far", e: ["start"] },
    near: { n: "near", e: ["intercept", "start"] },
    intercept: { n: "intercept", e: ["start"] },
  },
  edges: { /* Ребра автомата (имя каждого ребра указывает на
    узел-источник и узел-приемник) */
    start_close:
      [{ guard: [{ s: "lt", l: {v: "dist"}, r: 2 } ] }],
    /* Список guard описывает перечень условий, проверяемых
     * для перехода по ребру. Знак lt - меньше, lte - меньше
     * либо равно. В качестве параметров принимаются числа или
     * значения переменных "v" или таймеров "t" */
    start_near: [{ guard: [{ s: "lt", l: {v: "dist"}, r: 10 },
      { s: "lte", l: 2, r: {v: "dist"} } ] }],
    start_far:
      [{ guard: [{ s: "lte", l: 10, r: {v: "dist"} } ] }],
    close_catch: [{ synch: "catch!" }],
    /* Событие синхронизации synch вызывает на выполнение
     * соответствующую функцию */
    catch_kick: [{ synch: "kick!" }],
    kick_start: [{ synch: "goBack!", assign: [{ n: "t", v: 0,
      type: "timer" } ] }],
    /* Список assign перечисляет присваивания для переменных
     * "variable" и таймеров "timer" */
    far_start: [{ guard: [{ s: "lt", l: 10, r: {t: "t"} } ],
    synch: "lookAround!",
      assign: [{ n: "t", v: 0, type: "timer" } ] },
      { guard: [{ s: "lte", l: {t: "t"}, r: 10 } ],
        synch: "ok!" } ],
    near_start: [{ synch: "empty!",
      assign: [{ n: "t", v: 0, type: "timer" } ] } ],
    near_intercept: [{ synch: "canIntercept?" } ],
    /* Событие синхронизации synch может вызывать
```



```

    * соответствующую функцию для проверки возможности перехода
    * по ребру (заканчивается на знак "?") */
    intercept_start: [{ synch: "runToBall!", assign: [{ n: "t",
v: 0, type: "timer" }] }]
  },
  actions: {
    init(taken, state) { // Инициализация игрока
      state.local.goalie = true
      state.local.catch = 0
    },
    beforeAction(taken, state) {
      // Действие перед каждым вычислением
      if(taken.ball) state.variables.dist = taken.ball.dist
    },
    catch(taken, state) { // Ловим мяч
      if(!taken.ball) {
        state.next = true
        return
      }
      let angle = taken.ball.angle
      let dist = taken.ball.dist
      state.next = false
      if(dist > 0.5) {
        if(state.local.goalie) {
          if(state.local.catch < 3) {
            state.local.catch++
            return {n: "catch", v: angle}
          } else state.local.catch = 0
        }
        if(Math.abs(angle) > 15) return {n: "turn", v: angle}
        return {n: "dash", v: 20}
      }
      state.next = true
    },
    kick(taken, state) { // Пинаем мяч
      state.next = true
      if(!taken.ball) return
      let dist = taken.ball.dist
      if(dist > 0.5) return
      let goal = taken.goal
      let player = taken.teamOwn ? taken.teamOwn[0] : null
      let target
      if(goal && player)
        target = goal.dist < player.dist ? goal : player
      else
        if(goal) target = goal
      else
        if(player) target = player
      if(target) return
        {n: "kick", v: `${target.dist*2+40} ${target.angle}`}
      return {n: "kick", v: "10 45"}
    }
  }
}

```

```

    },
    goBack(taken, state) { // Возврат к воротам
      state.next = false
      let goalOwn = taken.goalOwn
      if(!goalOwn) return {n: "turn", v: 60}
      if(Math.abs(goalOwn.angle) > 10)
        return {n: "turn", v: goalOwn.angle}
      if(goalOwn.dist < 2) {
        state.next = true
        return {n: "turn", v: 180}
      }
      return {n: "dash", v: goalOwn.dist * 2 + 20}
    },
    lookAround(taken, state) { // Осматриваемся
      state.next = false
      state.synch = "lookAround!"
      if(!state.local.look)
        state.local.look = "left"
      switch (state.local.look) {
        case "left":
          state.local.look = "center";
          return {n: "turn", v: -60}
        case "center":
          state.local.look = "right"; return {n: "turn", v: 60}
        case "right":
          state.local.look = "back"; return {n: "turn", v: 60}
        case "back":
          state.local.look = "left"
          state.next = true
          state.synch = undefined
          return {n: "turn", v: -60}
        default: state.next = true
      }
    },
    canIntercept(taken, state) { // Можем добежать первыми
      let ball = taken.ball
      let ballPrev = taken.ballPrev
      state.next = true
      if(!ball) return false
      if(!ballPrev) return true
      if(ball.dist <= ballPrev.dist + 0.5) return true
      return false
    },
    runToBall(taken, state) { // Бежим к мячу
      state.next = false
      let ball = taken.ball
      if(!ball) return this.goBack(taken, state)
      if(ball.dist <= 2) {
        state.next = true
        return
      }
    }
  }

```



```

    }
    if(Math.abs(ball.angle) > 10)
        return {n: "turn", v: ball.angle}
    if(ball.dist < 2) {
        state.next = true
        return
    }
    return {n: "dash", v: 110}
},
ok(taken, state) { // Ничего делать не надо
    state.next = true; return {n: "turn", v: 0}
},
empty(taken, state) { state.next = true } // Пустое действие
}
}

```

В предложенной реализации у объекта выделено поле текущего состояния `current`, в котором хранится имя узла или ребра автомата, состояние `state`, список всех узлов `nodes`, список всех ребер `edges`, все возможные действия `actions`.

В состоянии `state` выделяются поля с переменными `variables`, таймерами `timers`, признаком перехода к следующему состоянию `next`, имени команды синхронизации, которая на предыдущем шаге не была завершена `synch`, объекта с локальными переменными автомата `local`, последние используются функциями для хранения дополнительной информации. Важно, что значения полей за исключением `local`, `next` и `synch` должны меняться только в менеджере, исполняющем временный автомат.

Все возможные узлы перечислены в `nodes`, ребра – в `edges`, действия – в `actions`. В числе действий выделяют `init`, `beforeAction`, `empty`. Первая вызывается при инициализации автомата, вторая – перед каждым действием, последняя – при отсутствии действия синхронизации, если это необходимо.

Менеджер временного автомата может выглядеть следующим образом:

```

const Taken = require('./taken')
const BEFORE_ACTION = "beforeAction"
const Manager = {
    setHear(input) { // Сохранение услышанного игроком
        Taken.setHear(input)
    },
    getAction(input, ta, team, side) { // Формирование действия
        let taken = Taken.setSee(input, team, side)
        this.incTimers(taken, ta)
        if(ta.actions[BEFORE_ACTION])
            ta.actions[BEFORE_ACTION](taken, ta.state)
        return this.execute(taken, ta)
    },
}

```

```

incTimers(taken, ta) { // Увеличение таймеров
  if(!this.lastTime)
    this.lastTime = 0
  if(taken.time > this.lastTime) {
    this.lastTime = taken.time
    for (let key in ta.state.timers)
      ta.state.timers[key] = ta.state.timers[key] + 1
  }
},
execute(taken, ta) { // Формирование действия
  if(ta.state.synch) { // Если действие выполнено не до конца
    let cond=ta.state.synch.substr(0,ta.state.synch.length-1)
    return ta.actions[cond](taken, ta.state)
  }
  if(ta.state.next) { // Переход на следующей действие
    if(ta.nodes[ta.current]) return this.nextState(taken, ta)
    if(ta.edges[ta.current]) return this.nextEdge(taken, ta)
  } // Переход не нужен
  if(ta.nodes[ta.current]) return this.executeState(taken, ta)
  if(ta.edges[ta.current]) return this.executeEdge(taken, ta)
  throw `Unexpected state: ${ta.current}`
},
nextState(taken, ta) { // Находимся в узле, нужен переход
  let node = ta.nodes[ta.current]
  for (let name of node.e) { // Перебираем ребра
    let edgeName = `${node.n}_${name}`
    let edge = ta.edges[edgeName]
    if(!edge) throw `Unexpected edge: ${node.n}_${name}`
    for (let e of edge) { // Проверяем все ребра
      if(e.guard) { // Проверяем ограничения
        let guard = true
        for (let g of e.guard)
          if(!this.guard(taken, ta, g)) {
            guard = false
            break // Ограничение не выполнено
          }
      }
      if(!guard) // Ребро нам не подходит
        continue
    }
  }
  if(e.synch) { // Необходима синхронизация
    if(e.synch.endsWith("?")) { // Проверка условия
      let cond = e.synch.substr(0, e.synch.length - 1)
      if(!ta.actions[cond])
        throw `Unexpected synch: ${e.synch}`
      console.log(`Synch[${taken.time}]: ${e.synch}`)
      if(!ta.actions[cond](taken, ta.state))
        continue // Проверка не успешна
    }
  }
  ta.current = edgeName // Далее работаем с этим ребром
  ta.state.next = false
}

```



```

        return this.execute(taken, ta) // Рекурсивный вызов
    }
},
nextEdge(taken, ta) { // Находимся в ребре, нужен переход
    let arr = ta.current.split("_")
    // После подчеркивания - имя узла, куда должны попасть
    let node = arr[1]
    ta.current = node
    ta.state.next = false
    return this.execute(taken, ta) // Рекурсивный вызов
},
executeState(taken, ta) { // Выполнить действия в узле
    let node = ta.nodes[ta.current]
    if(ta.actions[node]) { // Если действие в узле есть
        let action = ta.actions[node](taken, ta.state)
        if(!action && ta.state.next) return this.execute(taken, ta)
        return action
    }
    else { // Если действия в узле нет
        ta.state.next = true
        return this.execute(taken, ta) // Рекурсивный вызов
    }
},
executeEdge(taken, ta) { // Выполнить действия в ребре
    let edges = ta.edges[ta.current]
    for (let e of edges) { // Может быть несколько ребер
        if(e.guard) { // Выбираем "наше" ребро
            let guard = true
            for (let g of e.guard)
                if(!this.guard(taken, ta, g)) {
                    guard = false
                    break // Ограничение не выполнено
                }
            if(!guard) continue // Ребро нам не подходит
        }
        if(e.assign) { // Есть назначения в ребре
            for (let a of e.assign) {
                if(a.type == "timer") { // Для таймеров
                    if(!ta.state.timers[a.n])
                        throw `Unexpected timer: ${a}`
                    ta.state.timers[a.n] = a.v
                } else { // Для переменных
                    if(!ta.state.variables[a.n])
                        throw `Unexpected variable: ${a}`
                    ta.state.variables[a.n] = a.v
                }
            }
        }
    }
    if(e.synch) { // Необходима синхронизация
        if(!e.synch.endsWith("?") && !e.synch.endsWith("!"))

```

```

        throw `Unexpected synch: ${e.synch}`
    if(e.synch.endsWith("!")) { // Выполнение действия
        let cond = e.synch.substr(0, e.synch.length - 1)
        if(!ta.actions[cond])
            throw `Unexpected synch: ${e.synch}`
        // Выполнение action
        return ta.actions[cond](taken, ta.state)
    }
}
}
ta.state.next = true // Действий нет, переход к узлу
return this.execute(taken, ta) // Рекурсивный вызов
},
guard(taken, ta, g) { // Проверка условий
    function taStateObject(o, ta) { /* Получение значения
        таймера/переменной (g.l или g.r) */
        if(typeof o == "object") return
            o.v ? ta.state.variables[o.v] : ta.state.timers[o.t]
        else return o
    }
    function lt(ta, l, r) { // Проверка неравенства
        return taStateObject(l, ta) < taStateObject(r, ta)
    }
    // TODO Проверка условий
    throw `Unexpected guard: ${JSON.stringify(g)}`
},
}

```

В данном примере предполагается, что реализован объект Taken, удовлетворяющий описанным требованиям. В нем реализованы функция Taken.setSee(input, team, side) для вычисления параметров объектов (мяча, игроков, ворот) и функция Taken.setHear(input) для сохранения информации о том, что слышит игрок.

Представленная реализация обладает рядом недостатков, которые предлагается преодолеть в практической работе:

- не на каждом шаге формируется действие (пропуск действия – потеря темпа игры);
- при реализации lookAround не учитывается текущее положение игрока на поле (правильнее поворачиваться относительно предопределенных флагов);
- не реализовано сохранение состояния того, что игрок только что видел, но, отвернувшись, перестал видеть.

5.3. Вопросы для самопроверки

1. В чем отличие временных автоматов от конечных автоматов?

2. Какую реализацию предложено использовать при отладке временных автоматов?
3. В чем отличия «Симулятора» от «Конкретного симулятора»?
4. В чем отличия синхронизирующих сообщений, заканчивающихся восклицательными и вопросительными знаками?
5. Какие данные и в каком формате предлагается предоставлять игроку для обработки во временном автомате?
6. Каждый узел и каждое ребро временного автомата обладают специфическими свойствами, перечислите их.
7. В чем заключаются недостатки предложенной реализации временного автомата и как их преодолеть?

5.4. Задание

Необходимо разработать программу обработки временных автоматов и два временных автомата, имитирующие атакующего игрока и игрока, защищающего ворота. Необходимо продемонстрировать, что вероятность перехвата мяча выше, чем вероятность забивания гола (вероятности должны быть не нулевыми).

Практическая работа 6. КОМАНДНАЯ ИГРА

6.1. Цель и задачи

Целью работы является создание команды игроков и координация ее действий с учетом автономности каждого игрока.

Для достижения поставленной цели необходимо решить следующие задачи:

- создать программу игрока, обеспечивающую интерпретацию иерархического контроллера;
- создать контроллеры для всех игроков команды и вратаря;
- проверить командную игру одной команды против другой.

6.2. Основные теоретические сведения

Иерархический контроллер. Контроллер агента получает информацию о внешнем мире, добавляет к ней воспоминания, формирует новые воспоминания и определяет действия, которые будут выполнены агентом, т. е. на каждом шаге он должен определить, что ему нужно запомнить и какое действие следует выполнить. При линейной реализации возникает проблема

масштаба, когда реализация сложных (или составных) действий затруднена большим количеством примитивных действий, которые должны выполняться на каждом шаге. Данная проблема может быть решена за счет выделения N контроллеров (обозначим $c(i)$, $i=1..N$) со следующими свойствами (рис. 6.1):

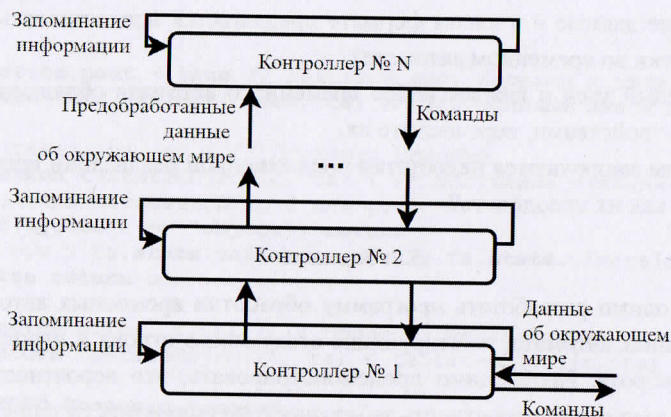


Рис. 6.1. Иерархический контроллер

1. Контроллер нижнего уровня $c(1)$ взаимодействует с окружающим миром.

2. Все контроллеры, кроме $c(N)$, взаимодействуют с контроллером верхнего уровня – передают предобработанные данные об окружающем мире
 $\text{send: } c(i) \rightarrow c(i+1), i=1..N-1$, получают команды от верхнего уровня
 $\text{rcv: } c(i+1) \rightarrow c(i), i=1..N-1$.

3. Все контроллеры имеют индивидуальные хранилища для результатов вычислений.

Использование иерархических контроллеров позволяет разделить логику и упростить проектирование агента, например, на нижних уровнях реализовывать реактивное поведение, которое зависит от ситуации, а на верхних уровнях обеспечить решение стратегических задач. Декомпозиция на контроллеры может строиться с использованием следующих подходов:

- структурная декомпозиция (контроллеры реализуют разные типы функций);
- декомпозиция на основе поведения агента (контроллеры реализуют различные сценарии поведения);
- декомпозиция по цели (контроллеры отвечают за достижение подцелей).

При этом применяют как чистые, так и смешанные подходы. С точки зрения игрока виртуального футбола можно предложить различные виды контроллеров в составе иерархического контроллера:

- структурная декомпозиция: перехват мяча, передача мяча, удар по воротам, ведение мяча и т. п.;
- декомпозиция на основе поведения агента: атака через центр, атака по краю, зонная защита, защита ворот и т. п.;
- декомпозиция по цели: расстановка игроков, перехват мяча, атака, защита и т. п.

Реализация иерархического контроллера. Рассмотрим пример трехуровневого иерархического контроллера для вратаря, который на нижнем уровне будет решать тактические задачи, а на верхнем уровне – стратегические.

Задачи контроллера нижнего уровня:

- сигнализация о возможности удара по мячу;
- передача команд управления агенту (движение, поворот, удар по мячу).

Задачи контроллера среднего уровня:

- движение в заданную точку (к центру ворот);
- обзор поля.

Задачи контроллера верхнего уровня:

- команда удара по мячу, когда он рядом;
- бросок к мячу и отбивание его от ворот.

При этом общая структура контроллера может выглядеть следующим образом:

```
const CTRL = {
  execute(input, controllers) {
    const next = controllers[0] // Следующий уровень
    let result = undefined
    // TODO Выполнение вычислений (result)
    if(next) { // Вызов следующего уровня
      const upper = next.execute(result, controllers.slice(1))
      // TODO Обработка ответа upper (обновление result)
    }
    return result
  }
}
module.exports = CTRL
```

Целесообразно создавать контроллеры в отдельных модулях, каждый контроллер должен принимать два параметра: входные данные для контрол-

лера и список следующих контроллеров. Если список не пуст, то следует вызывать следующий контроллер, передав ему результаты вычислений. Полученный ответ должен использоваться для формирования результата работы контроллера.

Для приведенного примера могут быть разработаны три контроллера.

1. Контроллер нижнего уровня может выглядеть следующим образом:

```
const Taken = require('./taken')
const CTRL_LOW = {
  execute(input, controllers) {
    const next = controllers[0] // Следующий уровень
    this.taken = Taken.setSee(input, this.team, this.side) //
    // Выделение объектов
    if(this.taken.ball && this.taken.ball.dist < 0.5) // Мяч рядом
      this.taken.canKick = true
    else
      this.taken.canKick = false
    if(next) // Вызов следующего уровня
      return next.execute(this.taken, controllers.slice(1))
  }
}
module.exports = CTRL_LOW
```

2. Контроллер среднего уровня может выглядеть следующим образом:

```
const CTRL_MIDDLE = {
  action: "return",
  turnData: "ft0",
  execute(input, controllers) {
    const next = controllers[0] // Следующий уровень
    switch (this.action) {
      case "return":
        input.cmd = this.actionReturn(input)
        break
      case "rotateCenter":
        input.cmd = this.rotateCenter(input)
        break
      case "seekBall":
        input.cmd = this.seekBall(input)
        break
    }
    input.action = this.action
    if(next) { // Вызов следующего уровня
      const command = next.execute(input, controllers.slice(1))
      if(command) return command
      if(input.newAction) this.action = input.newAction
      return input.cmd
    }
  },
  actionReturn(input) { // Возврат к своим воротам
```



```

    if(!input.goalOwn) return {n: "turn", v: 60}
    if(Math.abs(input.goalOwn.angle) > 10)
        return {n: "turn", v: input.goalOwn.angle}
    if(input.goalOwn.dist > 3)
        return {n: "dash", v: input.goalOwn.dist * 2 + 30}
    this.action = "rotateCenter"
    return {n: "turn", v: 180}
},
rotateCenter(input) { // Повернуться к центру
    if(!input.flags["fc"]) return {n: "turn", v: 60}
    this.action = "seekBall"
    return {n: "turn", v: input.flags["fc"].angle}
},
seekBall(input) { // Осмотр поля
    if(input.flags[this.turnData]) {
        if(Math.abs(input.flags[this.turnData].angle) > 10)
            return {n: "turn", v: input.flags[this.turnData].angle}
        if(this.turnData == "ft0") this.turnData = "fb0"
        else
            if(this.turnData == "fb0") {
                this.turnData = "ft0"
                this.action = "rotateCenter"
                return this.rotateCenter(input)
            }
    }
    if(this.turnData == "ft0")
        return {n: "turn", v: this.side == "1" ? -30 : 30}
    if(this.turnData == "fb0")
        return {n: "turn", v: this.side == "1" ? 30 : -30}
    throw `Unexpected state ${JSON.stringify(this)},
    ${JSON.stringify(input)}`
},
}
module.exports = CTRL_MIDDLE

```

3. Контроллер верхнего уровня может выглядеть следующим образом.

```

const CTRL_HIGH = {
    execute(input) {
        const immediate = this.immediateReaction(input)
        if(immediate) return immediate
        const defend = this.defendGoal(input)
        if(defend) return defend
        if(this.last == "defend")
            input.newAction = "return"
        this.last = "previous"
    },
    immediateReaction(input) { // Немедленная реакция
        if(input.canKick) {
            this.last = "kick"
            if(input.goal)
                return {n: "kick", v: `110 ${input.goal.angle}`}
            return {n: "kick", v: `10 45`}
        }
    }
}

```

```

    },
    defendGoal(input) { // Защита ворот
      if(input.ball) {
        const close = input.closest("b")
        if((close[0] && close[0].dist + 1 > input.ball.dist)
            || !close[0]) {
          this.last = "defend"
          if(Math.abs(input.ball.angle) > 5)
            return {n: "turn", v: input.ball.angle}
          return {n: "dash", v: 110}
        }
      }
    },
  },
}
module.exports = CTRL_HIGH

```

В данном примере можно проследить передачу информации между контроллерами разных уровней. Нижний уровень воспринимает информацию в том виде, в котором ее отдает среда моделирования, и вычисляет на ее основе параметры объектов (игроков, мяча, некоторых флагов), полученный результат передается в контроллер верхнего уровня, результат возвращается как команда агента.

Средний уровень по умолчанию пытается вернуться к центру ворот, а при нахождении в центре ворот осматривает поле, ориентируясь на два флага (ft0 и fb0). Результаты вычислений передаются контроллеру верхнего уровня.

Контроллер верхнего уровня анализирует признак «мяч рядом», при успешной проверке принимает решение ударить по мячу, если видны ворота противника, то ударить в сторону ворот, если не видны, то вбок. В случае если мяч далеко, то определить расстояние от мяча до ближайшего противника (оценить свои шансы перехватить мяч) и либо направиться к мячу, либо выполнить возврат к воротам.

К недостаткам представленной реализации следует отнести то, что никак не анализируется информация, поступающая от рефери (вбрасывание, гол и т. п.) и по-разному обрабатываются результаты работы контроллеров среднего и верхнего уровней. В рамках практических работ предлагается от данных недостатков избавиться.

6.3. Вопросы для самопроверки

1. Что такое иерархический контроллер?
2. Какие входные и выходные данные для каждого контроллера? В чем отличия для разных уровней контроллеров?

3. На основе каких подходов может формироваться иерархический контроллер?

4. В чем заключаются недостатки предложенной реализации иерархического контроллера и как их преодолеть?

6.4. Задание

Необходимо определить перечень задач, решаемых игроками и вратарем, разбить каждую из них на подзадачи, затем также проанализировать подзадачи и т. д. В результате следует описать деревья задач (подзадач) игрока и вратаря. Для каждого уровня дерева необходимо разработать соответствующий контроллер и объединить их все в иерархические контроллеры для игрока и вратаря. В результате запуска необходимого количества агентов от обеих команд (игроков и вратарей) необходимо продемонстрировать игру в футбол.

Важное ограничение: недопустимо, чтобы игроки «толпами» бегали за мячом. При необходимости допустимо создание уникальных контроллеров для игроков.

Список рекомендуемой литературы

Беляев С. А. Разработка игр на языке JavaScript: учеб. пособие для СПО. СПб.: Лань, 2020. 152 с. ISBN 978-5-8114-5881-3.

Жандаров В. В., Пантелеев М. Г. Мультиагентные системы. СПб.: Изд-во СПбГЭТУ «ЛЭТИ», 2016. 36 с.

Жандаров В. В., Пантелеев М. Г. Построение мультиагентных систем в среде JADE: учеб.-метод. пособие. СПб.: Изд-во СПбГЭТУ «ЛЭТИ», 2018. 36 с.

Рассел С., Норвиг П. Искусственный интеллект. Современный подход. 2-е изд. М.: Вильямс, 2018. 1408 с. ISBN 978-5-8459-1968-7.

Vidal Jose. Fundamentals of Multiagent Systems. 2007. 155 p.

Foundation for Intelligent Physical Agents (FIPA). URL: <http://www.fipa.org/>

Funge John, Millington Ian. Artificial Intelligence for Games. М.: Рид Элсивер, Morgan Kaufmann, 2009. 896 с. ISBN 978-0-12-374731-0.

Georgios N. Yannakakis and Julian Togelius: Artificial Intelligence and Games. М.: Springer, 2018. 360 с. Print ISBN: 978-3-319-63518-7, Online ISBN: 978-3-319-63519-4.

Java Agent Development Framework (JADE). URL: <http://jade.tilab.com/>

Users Manual RoboCup Soccer Server for Soccer Server Version 7.07 and later / Mao Chen, Klaus Dorer et al. 2003. 150 p.

Poole David, Mackworth Alan. Artificial Intelligence: Foundations of Computational Agents, second edition. Cambridge: Cambridge University Press, 2017. URL: <https://artint.info/>

Содержание

Введение.....	3
1. УСТАНОВКА И НАСТРОЙКА СРЕДЫ РАЗРАБОТКИ	4
2. ИНТЕРФЕЙС ВЗАИМОДЕЙСТВИЯ С СЕРВЕРОМ	6
3. ВЗАИМОДЕЙСТВИЕ С СЕРВЕРОМ НА JAVASCRIPT	14
4. КРАТКИЕ ОСНОВЫ МУЛЬТИАГЕНТНЫХ СИСТЕМ	18
Практическая работа 1. ОПРЕДЕЛЕНИЕ МЕСТОНАХОЖДЕНИЯ ИГРОКА НА ПОЛЕ	23
Практическая работа 2. ДВИЖЕНИЕ ИГРОКА ПО МАРШРУТУ	27
Практическая работа 3. УПРАВЛЕНИЕ НЕСКОЛЬКИМИ ИГРОКАМИ.....	29
Практическая работа 4. КООРДИНАЦИЯ ДЕЙСТВИЙ ИГРОКОВ.....	36
Практическая работа 5. РЕШЕНИЕ ЦЕЛЕВОЙ ЗАДАЧИ В УСЛОВИЯХ ПРОТИВОДЕЙСТВИЯ	39
Практическая работа 6. КОМАНДНАЯ ИГРА	53
Список рекомендуемой литературы.....	60

Сергей Алексеевич Беляев

**Интеллектуальные системы
Программирование игроков в виртуальном футболе**

Лабораторный практикум

Редактор И. Г. Скачек

Подписано в печать 24.12.20. Формат 60×84 1/16.
Бумага офсетная. Печать цифровая. Печ. л. 4,0.
Гарнитура «Times New Roman». Тираж 75 экз. Заказ 178.

Издательство СПбГЭТУ «ЛЭТИ»
197376, С.-Петербург, ул. Проф. Попова, 5